

PLAN.MD — Déroulé de la démo EnerFlex (≈ 35 min)

***Le message de la démo** : sur une application existante, une équipe d'agents IA livre une user story de bout en bout — mais **l'humain relit et valide trois fois** : le plan, les tests, le code. L'IA va vite ; la relecture fait la qualité. Tout se termine par un rapport de livraison et une spécification technique à jour, sans qu'on les ait écrits à la main.*

0. Avant d'entrer en salle (10 min, la veille)

```
cd C:\dev\workspaces\ai-demo-2026\demo3
npm install
npm test                # tout vert, couverture > 70 % des deux côtés
npm run dev              # API :3001 + UI :5173
```

Ouvrir <http://localhost:5173> et vérifier :

- le tableau de bord d'**août 2026** (mois par défaut), avec ses alertes ;
- la fiche **LIL-02 — Entrepôt de Lesquin** : la jauge de puissance reste très basse (site surdimensionné) ;
- la fiche **LYO-01 — Usine de Vénissieux** : dépassements, alerte Critique, ligne de pénalité non nulle (site sous-dimensionné).

Ce sont les deux sites qu'on regardera après la story : l'un doit recommander de **réduire**, l'autre d'**augmenter** la puissance souscrite.

Puis, dans Claude Code (ouvert dans `demo3/`) :

- vérifier que les agents apparaissent : `/agents` → 7 agents ;
- vérifier que les skills apparaissent : taper `/` → `implement-us`, `story-readiness`, ... ;
- vérifier que le serveur MCP Chrome DevTools est connecté (`/mcp`) — l'agent `reviewer` et `react-dev` s'en servent pour voir l'écran ;
- `git status` propre, `git log -1` noté : c'est le point de retour si tout doit être rejoué (`git reset --hard <commit>`).

Prévoir deux fenêtres côte à côte : Claude Code à gauche, le navigateur (l'app, puis les documents Markdown) à droite.

1. Poser le décor (3 min)

À l'écran : l'application. À dire, en trois phrases :

1. « Voici EnerFlex : un client industriel, 12 sites, sa consommation, sa facture calculée depuis la grille du contrat — heures pleines / heures creuses, abonnement à la puissance souscrite, pénalités de dépassement — et ses alertes. » (montrer LYO-01 : la pénalité.)
2. « L'application existe déjà, avec ses tests, sa spec technique et ses règles de développement. » (montrer `docs/SPEC-TECHNIQUE.md` §4.3, la formule de la facture.)
3. « On va lui ajouter une fonctionnalité que le client vient de demander. » (ouvrir `specs/US-006-optimisation-puissance-souscrite.md`, lire le contexte métier et l'exemple.)

Puis montrer `.claude/agents/` : sept fichiers Markdown, « ce sont les fiches de poste de l'équipe ». Ouvrir `business-analyst.md` dix secondes : un rôle, un périmètre, des interdits.

2. Lancer la story (le prompt unique)

Dans Claude Code :

```
Utilise le skill implement-us sur US-006.
```

Le skill enchaîne les agents et **s'arrête à chaque porte humaine**. Le déroulé ci-dessous dit, pour chaque étape, ce qui se passe, ce qu'on montre, et ce qu'on répond.

Variante étape par étape (si l'on préfère lancer chaque agent soi-même) : les prompts sont en annexe A.

3. Étape par étape

3.1 — L'agent BA relit la story (≈ 3 min)

Ce qui se passe : `business-analyst` lit la story, la spec technique et le code, et rend `docs/revues/US-006-revue.md` avec un verdict et des questions.

Ce qu'on montre : le rapport, côté navigateur. Les six axes, puis les constats. Lire les questions **à voix haute** : c'est le moment où le public comprend que l'agent *lit* au lieu d'obéir.

Les manques laissés volontairement dans la story (l'agent devrait en trouver au moins deux ; s'il en trouve d'autres, tant mieux — répondre de la même façon) :

Manque	Question probable	Réponse à donner
Le pas des puissances candidates n'est pas dit (l'exemple suggère 10 kVA sans l'énoncer)	« Quelles puissances sont candidates ? »	Multiples de 10 kVA , de 10 kVA jusqu'à la puissance maximale relevée du mois arrondie à la dizaine supérieure.
Égalité de coût entre deux puissances non tranchée	« Laquelle retenir à coût égal ? »	La plus élevée (prudence : moins de dépassements).
Cas « déjà optimale » absent (CA-5 ne prévoit que réduire / augmenter)	« Que montrer si la puissance actuelle est déjà la meilleure ? »	Économie 0,00 € , sens « Conserver la puissance souscrite » — on accepte d'ajouter ce troisième libellé à CA-5.
Arrondi des coûts intermédiaires	« Arrondir chaque coût ou seulement l'économie ? »	Calcul sans arrondi , arrondi au centime à l'affichage de chaque montant.

Ce qu'on répond : une ligne par question, dans la conversation. Dire au public : « Ces réponses font partie du besoin ; elles seront transmises à l'architecte et reportées dans la story par un humain. »

3.2 — L'agent architecte écrit le plan → II porte humaine n° 1 (≈ 4 min)

Ce qui se passe : `architect` produit `docs/architecture/US-006-plan.md` : la règle comme fonction pure (`recommanderPuissance`), l'exemple chiffré recalculé, la route (`GET /api/sites/:id/optimisation?mois=`), le type de réponse, le bloc d'écran, la **liste nominative des tests**, deux alternatives, un ADR si besoin.

Ce qu'on montre : le plan dans le navigateur. Insister sur :

- §2 : la règle tient en une signature et un exemple — « c'est ça qu'on relit, pas du code » ;
- §5 : le plan de tests — « chaque critère a déjà son test, nommé en français » ;
- §6 : les alternatives — « l'agent ne présente jamais une seule option ».

Ce qu'on fait : relire vraiment, puis répondre « Le plan est validé. » Astuce : demander **une** modification pour montrer la boucle — par exemple « ajoute un test pour un mois sans aucun relevé au-dessus de 10 kVA » — puis valider.

3.3 — L'agent test-dev écrit les tests, rouges → II porte humaine n° 2 (≈ 4 min)

Ce qui se passe : `test-dev` écrit `backend/tests/optimisation.test.ts`, complète `api.test.ts`, ajoute les tests d'UI ; il lance les suites : les nouveaux tests **échouent** (module absent), les anciens passent.

Ce qu'on montre : un fichier de test. Les noms en français se lisent comme la story ; les valeurs attendues sont **calculées à la main en commentaire** (« $P = 90 : 256,50 + 71,25 = 327,75$ »). Puis le terminal : « 6 failed, 48 passed » — « le rouge est le résultat attendu ».

Ce qu'on fait : relire un ou deux tests, puis « Les tests sont validés. »

3.4 — Les agents dev implémentent, en parallèle (≈ 6 min)

Ce qui se passe : `node-dev` (domaine → service → route) et `react-dev` (type → api → hook → composant → page) tournent **en même temps**, chacun dans son dossier, jusqu'au vert. `react-dev` finit par ouvrir l'écran dans Chrome via MCP.

Ce qu'on montre : pendant que ça tourne, revenir sur les règles (`.claude/rules/`) : « un agent, un dossier ; les tests ne sont jamais modifiés ; la règle vit dans `domain/` ». À la fin : `npm test` vert des deux côtés, couverture affichée.

3.5 — L'agent reviewer relit → II porte humaine n° 3 (≈ 4 min)

Ce qui se passe : `reviewer` (lecture seule) rend `docs/revues/US-006-revue-code.md` : critères couverts, fidélité au plan, tests non affaiblis, règles, écran vu dans Chrome, constats par sévérité.

Ce qu'on montre : le rapport ; puis `git diff --stat` ; puis **l'application** : `/sites/LIL-02` → « Réduire la puissance souscrite » (630 → 250 kVA, ≈ 1 083 € HT d'économie par mois) ; `/sites/LY0-01` → « Augmenter la puissance souscrite » (1 250 → 1 480 kVA, ≈ 5 038 € HT par mois : les pénalités disparaissent presque).

Point de discussion si le public le remarque : avec cette grille (pénalité 0,95 €/kWh contre abonnement 2,85 €/kVA), l'optimum tolère environ trois heures de dépassement par mois — tous les sites reçoivent donc une recommandation légèrement sous leur point. C'est le contrat qui le veut, pas l'algorithme.

Ce qu'on fait : relire un fichier de code (le domaine, `optimisation.ts` : une vingtaine de lignes), puis « Le code est validé. »

3.6 — L'agent tech-writer clôture (≈ 3 min)

Ce qui se passe : `tech-writer` rend `docs/rapports/US-006-rapport.md` (+ PDF) et met à jour `docs/SPEC-TECHNIQUE.md` (§4 nouvelle règle, §5 nouvelle route, §6 nouveau bloc, §9 historique, version 1.1). Puis commit.

Ce qu'on montre : le PDF du rapport (le tableau du parcours, avec les trois portes humaines datées) et le diff de la spec technique.

4. Conclure (2 min)

- Sept agents, trois relectures humaines, une story livrée avec ses tests, sa revue, son rapport et sa spec — **en 30 minutes**, sur une base existante.
- Ce qui a fait la qualité : la revue du besoin avant le code, les tests avant le code, un relecteur qui ne corrige pas, et un humain qui dit « oui » trois fois.
- Tout est dans le dépôt : les fiches de poste (`.claude/agents/`), les pratiques (`.claude/skills/`), les règles (`.claude/rules/`). « Ce n'est pas de la magie : c'est de l'organisation, écrite en Markdown. »

5. Si quelque chose tourne mal

Problème	Parade
Le BA rend « Prête » sans question	Dire que la story était bien écrite, et provoquer la boucle à la porte n° 1 (demander une modification du plan).
Un agent dépasse son dossier	Le dire au public : « c'est exactement ce que le reviewer va relever » — et laisser le reviewer le trouver.
Les devs dépassent 8 min	Continuer à parler des règles et des skills ; le résultat arrive.
Vite affiche un autre port que 5173	Un autre projet occupe 5173 : utiliser le port affiché dans le terminal (5174 ...), le proxy <code>/api</code> fonctionne quel que soit le port.
Un port est occupé	<code>Get-NetTCPConnection -LocalPort 3001,5173 Select OwningProcess</code> puis <code>Stop-Process -Id <pid></code> .
Tout rejouer	<code>git reset --hard <commit noté en §0></code> puis relancer <code>npm run dev</code> .
Plus que 15 minutes	Jouer les étapes 3.1 à 3.3 seulement (revue, plan, tests rouges) : c'est déjà le cœur du message — relire avant de coder.

Annexe A — Prompts étape par étape

1. Revue du besoin

Utilise le skill story-readiness sur US-006 : délègue à l'agent business-analyst.

2. Plan

Utilise le skill architecture-plan pour US-006 : délègue à l'agent architect avec docs/revues/US-006-revue.md et mes réponses : (1) multiples de 10 kVA jusqu'à la puissance max relevée arrondie à la dizaine supérieure ; (2) à coût égal, la plus élevée ; (3) si déjà optimale : économie 0,00 € et « Conserver la puissance souscrite » ; (4) calcul sans arrondi, arrondi au centime à l'affichage.

3. Tests

Lance l'agent test-dev sur US-006 avec docs/architecture/US-006-plan.md : uniquement des fichiers de test, suite rouge pour la bonne raison. Puis liste-moi les tests écrits.

4. Implémentation

Lance en parallèle node-dev (backend/**) et react-dev (frontend/**) sur US-006 avec le plan et les tests rouges. Ils ne modifient pas les tests. Puis npm run typecheck && npm run lint && npm test.

5. Revue

Lance l'agent reviewer sur US-006 avec le plan et le commit de départ <sha>.

6. Clôture

Lance l'agent tech-writer sur US-006 : rapport de livraison (md + PDF) et mise à jour de docs/SPEC-TECHNIQUE.md. Puis commit « feat(enerflex): US-006 recommandation de puissance souscrite ».

Annexe B — Pourquoi cette story

- **Reconnaissable en cinq secondes** : tout le monde dans la salle sait ce qu'est une puissance souscrite et une pénalité de dépassement.
- **Une vraie règle métier** (un petit algorithme d'optimisation), donc de vrais tests de domaine avec des valeurs calculées à la main — le TDD a quelque chose à mordre.
- **Une tranche complète mais courte** : une fonction de domaine, une route, un bloc d'écran. Pas de nouvelle entité, pas de migration, pas de formulaire.
- **Deux résultats opposés visibles à l'écran** (LIL-02 réduire, LYO-01 augmenter) : la démo finit sur l'application, pas sur un terminal.
- **Des manques volontaires** dans la story, pour que le BA ait quelque chose à dire.